PROGRAMACIÓN AVANZADA

Hibernate

Parte II



Ing. Matias Macrino

HIBERNATE

Tipos Básicos:

- ✔Al mapear una variable de instancia Java a una columna en la BD, se puede especificar el tipo
- ✔Algunos tipos de datos nativos Java pueden mapearse de distintas formas a la BD.
- ✔***Hibernate*** nos permite especificar un tipo mas concreto para cada dato Java.

HIBERNATE

Tipos Básicos:

Tipo Hibernate	Tipo Java, Wrapper	Tipo base de datos (MSSQL/MySQL)
integer	int , java.lang.Integer	INT/INTEGER
long	long, java.lang.Long	BIGINT/BIGINT
float	float , java.lang.Float	REAL/FLOAT
double	double , java.lang.Double	DOUBLE/DOUBLE
character	char , java.lang.Character	CHAR/CHAR
boolean	boolean , java.lang.Boolean	BIT/TINYINT. Guarda el true como "1" y el false como "0"
yes_no	boolean , java.lang.Boolean	CHAR(1). Guarda el true como "Y" y el false como "N"
true_false	boolean , java.lang.Boolean	CHAR(1). Guarda el true como "T" y el false como "F"
string	java.lang.String	VARCHAR
text	java.lang.String	TEXT/LONGTEXT

HIBERNATE

Tipos Básicos:

Tipo Hibernate	Tipo Java, Wrapper	Tipo base de datos (MSSQL/MySQL)
binary	byte[]	BINARY/TINYBLOB
big_decimal	java.math.BigDecimal	DECIMAL/DECIMAL
big_integer	java.math.BigInteger	DECIMAL/DECIMAL
date	java.util.Date	DATE/DATE. Solo se guarda solo la información de la fecha (año, mes y día)
time	java.util.Date	TIME/TIME. Solo se guarda solo la información de la hora (horas, minutos y segundos)
timestamp	java.util.Date	DATETIME/DATETIME. Se guarda la información de la fecha y la hora (año, mes , día, horas, minutos y segundos)

HIBERNATE

Tipos Básicos:

```
1   @Type(type="integer")
2   private int inte;
3   @Type(type="long")
4   private long long1;
5   @Type(type="float")
6   private float float1;
7   @Type(type="double")
8   private double double1;
9   @Type(type="character")
10  private char character1;
11  @Type(type="boolean")
12  private boolean boolean1;
13  @Type(type="yes_no")
14  private boolean yesno1;
15  @Type(type="true_false")
16  private boolean truefalse1;
17  @Type(type="string")
18  private String stri;
19  @Temporal(TemporalType.DATE)
20  private Date dateDate;
21  @Temporal(TemporalType.TIME)
22  private Date dateTime;
23  @Temporal(TemporalType.TIMESTAMP)
24  private Date dateTimestamp;
25  @Lob
26  private String texto;
27  @Type(type="binary")
28  private byte[] binario;
29  @Type(type="big_decimal")
30  private BigDecimal bigDecimal;
31  @Type(type="big_integer")
32  private BigInteger bigInteger;
33
```

HIBERNATE

Relaciones:

✔A continuación se explicara como utilizar las distintas relaciones entre entidades ***Hibernate.***

✔Tipos de relaciones:

- Uno a Uno (unidireccional)
- Uno a Uno (bidireccional)
- Uno a Muchos (desordenada)
- Uno a Muchos (ordenada)
- Muchos a Muchos

HIBERNATE

Relaciones:

✔Ejemplos para cada tipo de relaciones:

- ***Uno a Uno:*** Una entidad ***Profesor*** a una entidad ***Direccion***
- ***Uno a Muchos:*** Una entidad ***Profesor*** a muchos ***Correo Electronicos***.
- ***Muchos a Muchos:*** Muchas entidades ***Profesor*** a muchas entidades ***Modulo***.

HIBERNATE

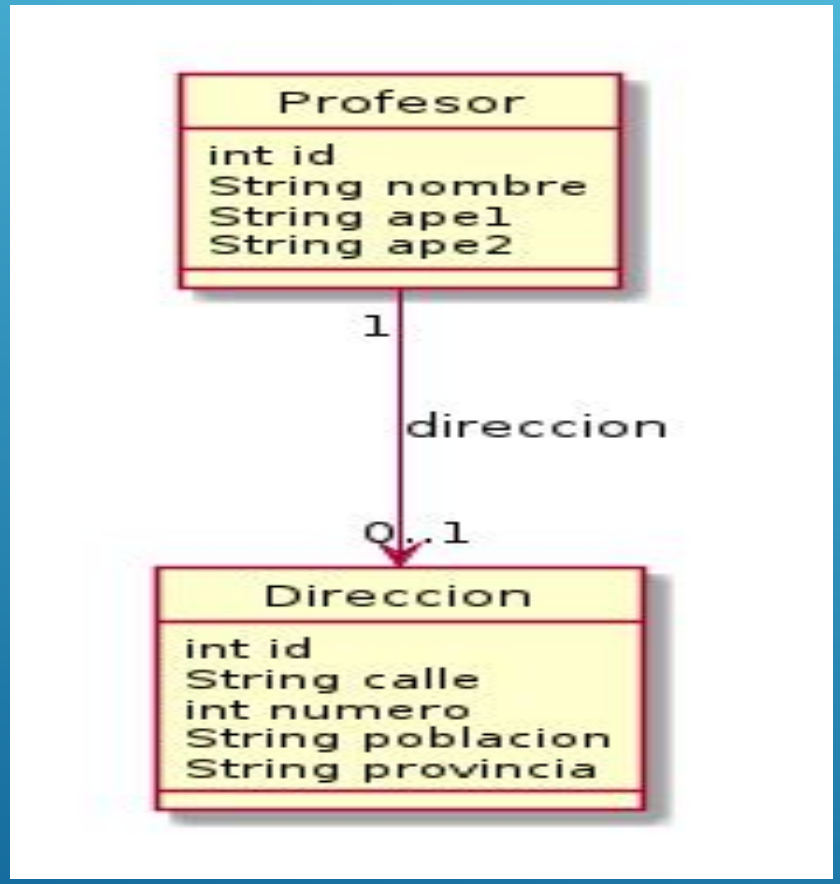
Relaciones Uno a Uno (unidireccional):

- ✔La relación de Uno a Uno en ***Hibernate*** consiste simplemente en que un objeto tengo una referencia a otro objeto de forma que al persistir el primer objeto también se haga lo mismo con el segundo

- ✔Con unidireccional se refiere a que la relación uno a uno va a ser en un único sentido.

HIBERNATE

Relaciones Uno a Uno (unidireccional):



HIBERNATE

Relaciones Uno a Uno (unidireccional):

```java
@Entity
@Table(name="Direccion")
public class Direccion implements Serializable  {

    @Id
    @Column(name="Id")
    private int id;

    @Column(name="calle")
    private String calle;

    @Column(name="numero")
    private int numero;

    @Column(name="poblacion")
    private String poblacion;

    @Column(name="provincia")
    private String provincia;

}
```

HIBERNATE

Relaciones Uno a Uno (unidireccional):

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;
    @Column(name="nombre")
    private String nombre;
    @Column(name="apellido_uno")
    private String ape1;
    @Column(name="apellido_dos")
    private String ape2;
    @OneToOne(cascade=CascadeType.ALL)
    private Direccion direccion;

}

```

@OneToOne: Anotación de JPA. Esta anotación indica que es una relación uno a uno de las dos tablas en cuestión. El atributo "cascade" indicar a ***Hibernate*** como debe actuar cuando realiza operaciones de persistencia.

HIBERNATE

Relaciones Uno a Uno (unidireccional):

```
1   Direccion direccion = new Direccion(1, "12 de Octubre", 456, "Martinez", "Bs. As");
2   Profesor profesor = new Profesor(1, "Juan", "Perez", "");
3   profesor.setDireccion(direccion);
4
5   Session session = sessionFactory.openSession();
6   session.beginTransaction();
7
8   session.save(profesor);
9
10  session.getTransaction().commit();
11  session.close();
12
```

Para la persistencia solamente se crean las dos clases correspondientes, líneas 1 y 2, asignando el objeto referenciado al objeto a persistir, línea 3.

Al ejecutarse ***hibernate,*** lo que realiza es la persistencia de dos registros en la BD, cuando desde Java solo se envió un solo objeto.

Primero realizara un ***SELECT*** contra ***Direcciones*** para comprobar su existencia. Si ya existe no insertara una nueva fila, sino que hará un ***UPDATE.***

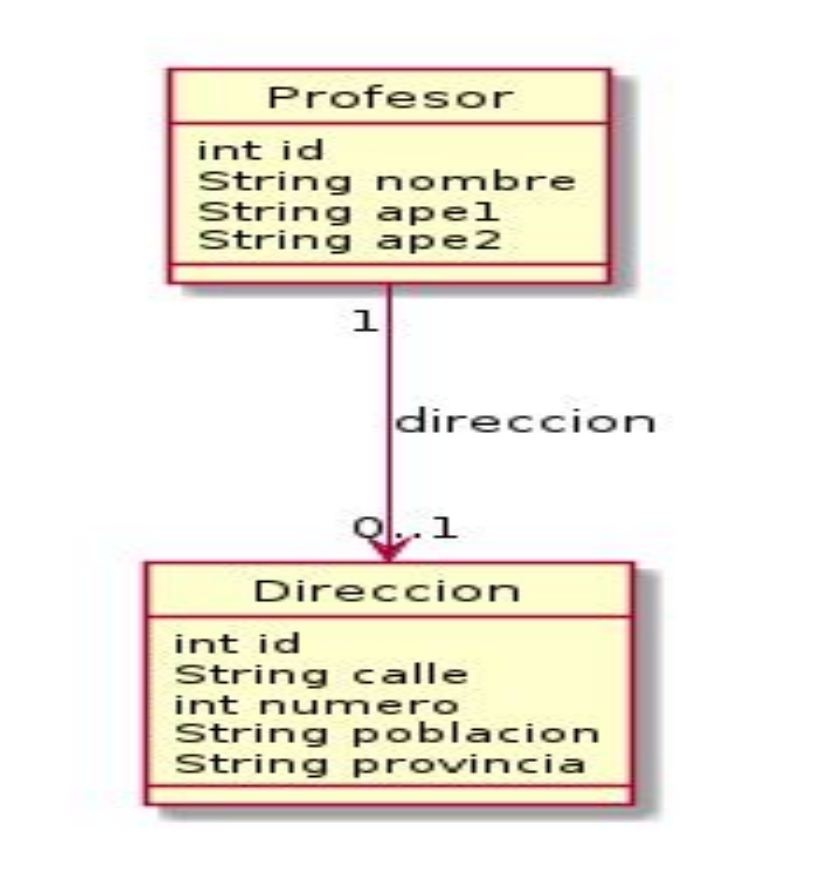
HIBERNATE

Relaciones Uno a Uno (bidireccional):

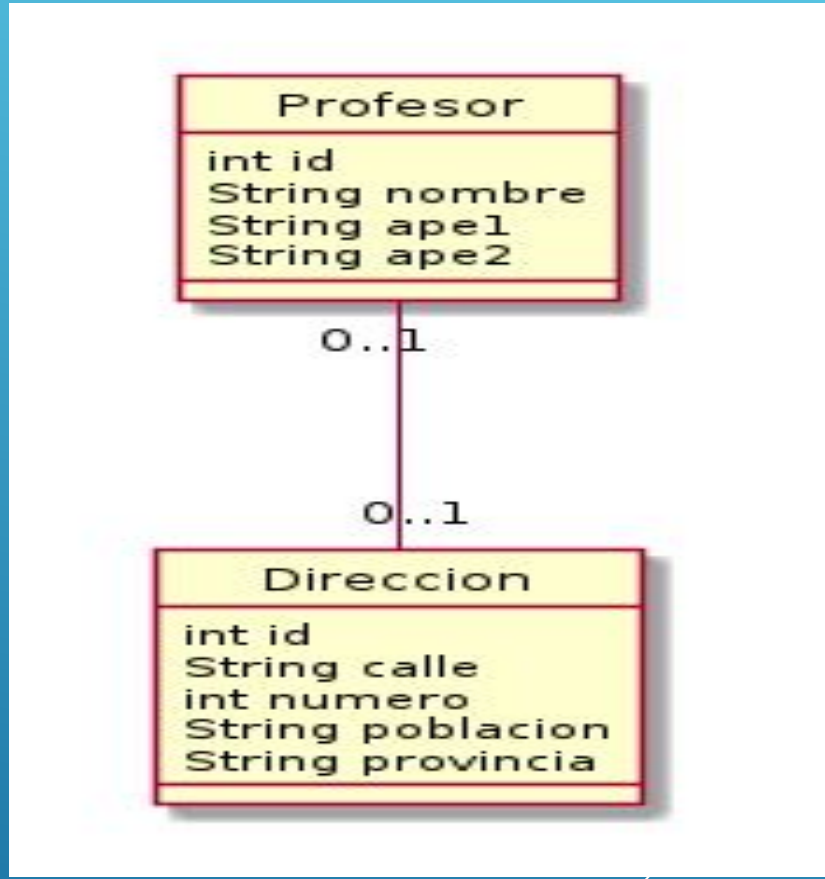
- ✔Es igual a una relación Uno a Uno Unidireccional. Pero en este caso la relación entre las clases va ser en sentido bidireccional.

HIBERNATE

Relaciones Uno a Uno (bidireccional):



Unidireccional



Bidireccional

HIBERNATE

Relaciones Uno a Uno (bidireccional):

```java
@Entity
@Table(name="Direccion")
public class Direccion implements Serializable  {

    @Id
    @Column(name="Id")
    private int id;

    @Column(name="calle")
    private String calle;

    @Column(name="numero")
    private int numero;

    @Column(name="poblacion")
    private String poblacion;

    @Column(name="provincia")
    private String provincia;

}
```

Unidireccional

```java
@Entity
@Table(name="Direccion")
public class Direccion implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="calle")
    private String calle;

    @Column(name="numero")
    private int numero;

    @Column(name="poblacion")
    private String poblacion;

    @Column(name="provincia")
    private String provincia;

    @OneToOne(cascade=CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Profesor profesor;
}
```

Bidireccional

HIBERNATE

Relaciones Uno a Uno (bidireccional):

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;
    @Column(name="nombre")
    private String nombre;
    @Column(name="apellido_uno")
    private String ape1;
    @Column(name="apellido_dos")
    private String ape2;
    @OneToOne(cascade=CascadeType.ALL)
    private Direccion direccion;

}

```

Unidireccional

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="nombre")
    private String nombre;

    @Column(name="apellido_uno")
    private String ape1;

    @Column(name="apellido_dos")
    private String ape2;

    @OneToOne(cascade=CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Direccion direccion;
}

```

Bidireccional

HIBERNATE

Relaciones Uno a Uno (bidireccional):

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="nombre")
    private String nombre;

    @Column(name="apellido_uno")
    private String ape1;

    @Column(name="apellido_dos")
    private String ape2;

    @OneToOne(cascade=CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Direccion direccion;
}
```

```java
@Entity
@Table(name="Direccion")
public class Direccion implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="calle")
    private String calle;

    @Column(name="numero")
    private int numero;

    @Column(name="poblacion")
    private String poblacion;

    @Column(name="provincia")
    private String provincia;

    @OneToOne(cascade=CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Profesor profesor;
}
```

@PrimaryKeyJoinColumn: Esta anotación indica que la relación entra las dos tablas se realiza mediante la ***PK (Primary Key).***

HIBERNATE

Relaciones Uno a Uno (bidireccional):

```
1   Direccion direccion1=new Direccion(3, "Independencia", 736, "Pilar", "Bs. As");
2   Profesor profesor1=new Profesor(3, "Sergio", "Mateo", "Gomez");
3   profesor1.setDireccion(direccion1);
4   direccion1.setProfesor(profesor1);
5
6   Direccion direccion2=new Direccion(4, "La Madrid", 454, "Moron", "Bs. As");
7   Profesor profesor2=new Profesor(4, "Juan", "Moreno", "Díaz");
8   profesor2.setDireccion(direccion2);
9   direccion2.setProfesor(profesor2);
10
11
12  Session session=sessionFactory.openSession();
13  session.beginTransaction();
14
15  session.save(profesor1);
16  session.save(direccion2);
17
18  session.getTransaction().commit();
19  session.close();
20
```

En las líneas 3 y 4 se crea la relación entre Dirección y Profesor. Lo cual se persiste en la línea 15.
En las líneas 8 y 9 se crea la relación entre Dirección y Profesor. Lo cual se persiste en la línea 16.
En ambos casos el resultado es el mismo. Pero no son iguales.
En el primer caso, al persistir Profesor se inserta dicho registro por lo que no puede existir pero si permite que la dirección exista.
En el segundo caso, al persistir Dirección ocurre lo contrario no podrá existir un registro dirección pero si podría existir el profesor.

HIBERNATE

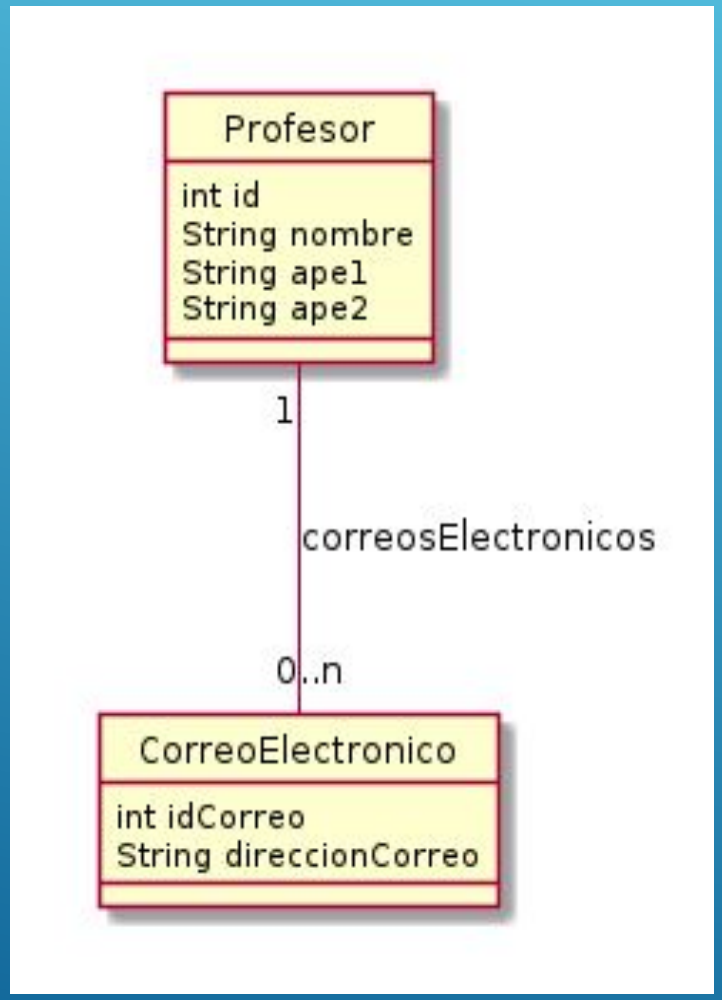
Relaciones Uno a Muchos (desordenada):

✔Dicha relación consiste simplemente en que un ***objeto "padre"*** tenga una lista ***sin ordenar*** de otros ***objetos "hijos"*** de forma que al persistir el objeto principal también se persistirá la lista de ***objetos "hijos"*** **.**

✔También esta relación suele llamarse padre-hijo.

HIBERNATE

Relaciones Uno a Muchos (desordenada):



HIBERNATE

Relaciones Uno a Muchos (desordenada):

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="nombre")
    private String nombre;

    @Column(name="apellido_uno")
    private String ape1;

    @Column(name="apellido_dos")
    private String ape2;

    @OneToMany(mappedBy="profesor",cascade= CascadeType.ALL)
    private Set<CorreoElectronico> correosElectronicos;

}

```

@OneToMany: Indica que es una relación uno a muchos. Como su nombre lo indica le esta informando a ***Hibernate*** que esta propiedad contendrá la lista de objetos hijos.

mappedBy: Contendrá el nombre de la propiedad Java de la clase hija que enlaza con la clase padre.

- Se utiliza la interfaz Set, ya que nos permite armar una lista desordenada de objetos.

HIBERNATE

Relaciones Uno a Muchos (desordenada):

```java
@Entity
@Table(name="CorreoElectronico")
public class CorreoElectronico implements Serializable {

    @Id
    @Column(name="IdCorreo")
    private int idCorreo;

    @Column(name="DireccionCorreo")
    private String direccionCorreo;

    @ManyToOne
    @JoinColumn(name="IdProfesor")
    private Profesor profesor;

}
```

@ManyToOne: Indica que es una relación muchos a uno. Como se encuentra del otro lado de la relación, se debe de indicar a ***Hibernate***.

JoinColumn: Se indica el nombre de la columna que en la tabla *hija* contiene la ***FK*** a la tabla *padre*.

HIBERNATE

Relaciones Uno a Muchos (desordenada):

```
1   Profesor profesor = new Profesor(8, "Sara", "Barrrera", "");
2   Set<CorreoElectronico> correosElectronicos = new HashSet<>();
3   correosElectronicos.add(new CorreoElectronico(3, "sara@yahoo.com",profesor));
4   correosElectronicos.add(new CorreoElectronico(2, "sara@hotmail.com",profesor));
5   correosElectronicos.add(new CorreoElectronico(1, "sara@gmail.com",profesor));
6
7   profesor.setCorreosElectronicos(correosElectronicos);
8
9   Session session=sessionFactory.openSession();
10  session.beginTransaction();
11
12  session.save(profesor);
13
14  session.getTransaction().commit();
15  session.close();
16
```

- En la línea 2, utilizamos la implementación HashSet de la interfaz Set.
- En la línea 7 se establece la relación entre los objetos hijos al padre. Para este caso CorreoElectronico a Profesor.
- En la línea 12 se persiste el objeto padre (Profesor) y automáticamente se hace lo mismo con los objetos hijos (CorreoElectronico).

HIBERNATE

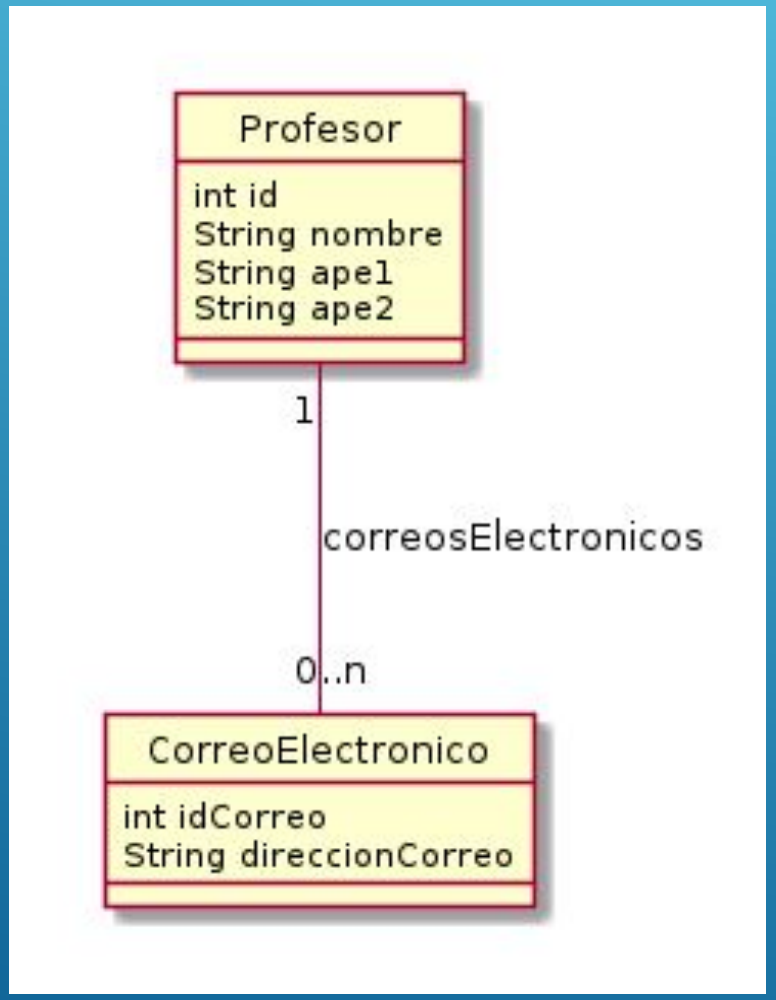
Relaciones Uno a Muchos (ordenada):

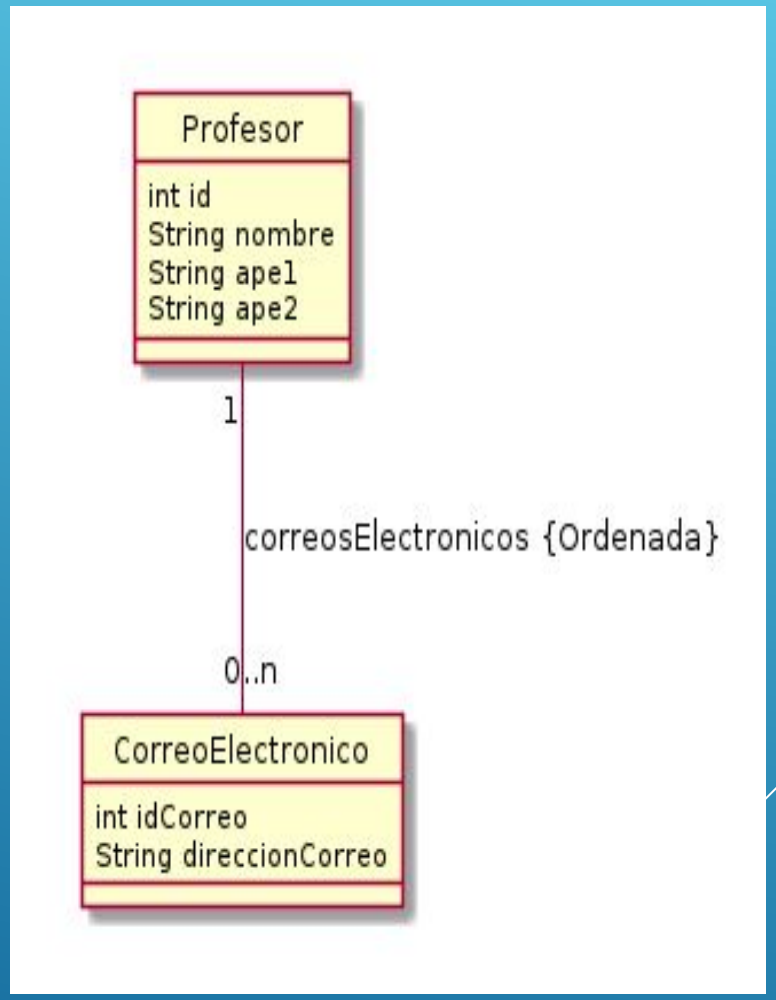
- ✔Es igual a la relación ***Uno a Muchos desordenada***, con la única diferencia que el objeto padre tendrá un ***lista ordena*** de los objetos hijos.

- ✔También esta relación suele llamarse padre-hijo.

HIBERNATE

Relaciones Uno a Muchos (ordenada):





HIBERNATE

Relaciones Uno a Muchos (ordenada):

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="nombre")
    private String nombre;

    @Column(name="apellido_uno")
    private String ape1;

    @Column(name="apellido_dos")
    private String ape2;

    @OneToMany(cascade= CascadeType.ALL)
    @JoinColumn(name="IdProfesor")
    @IndexColumn(name="idx")
    private List<CorreoElectronico> correosElectronicos;

}
```

@IndexColumn: Indicaremos el nombre de la columna que en la tabla hija contiene el orden dentro de la lista de hijos. En nuestro ejemplo es la columna de la base de datos idx que se encuentra en la tabla CorreoElectronico.

HIBERNATE

Relaciones Uno a Muchos (desordenada):

```
1   Profesor profesor=new Profesor(9, "Rosa", "Díaz", "");
2   List<CorreoElectronico> correosElectronicos=new ArrayList<>();
3   correosElectronicos.add(new CorreoElectronico(3, "rosa@yahoo.com",profesor));
4   correosElectronicos.add(new CorreoElectronico(2, "rosa@hotmail.com",profesor));
5   correosElectronicos.add(new CorreoElectronico(1, "rosa@gmail.com",profesor));
6
7   profesor.setCorreosElectronicos(correosElectronicos);
8
9   Session session=sessionFactory.openSession();
10  session.beginTransaction();
11
12  session.save(profesor);
13
14  session.getTransaction().commit();
15  session.close();
16
```

- En la línea 2, utilizamos la implementación ArrayList de la interfaz List.
- En la línea 7 se establece la relación entre los objetos hijos al padre. Para este caso CorreoElectronico a Profesor.
- En la línea 12 se persiste el objeto padre (Profesor) y automáticamente se hace lo mismo con los objetos hijos (CorreoElectronico).

HIBERNATE

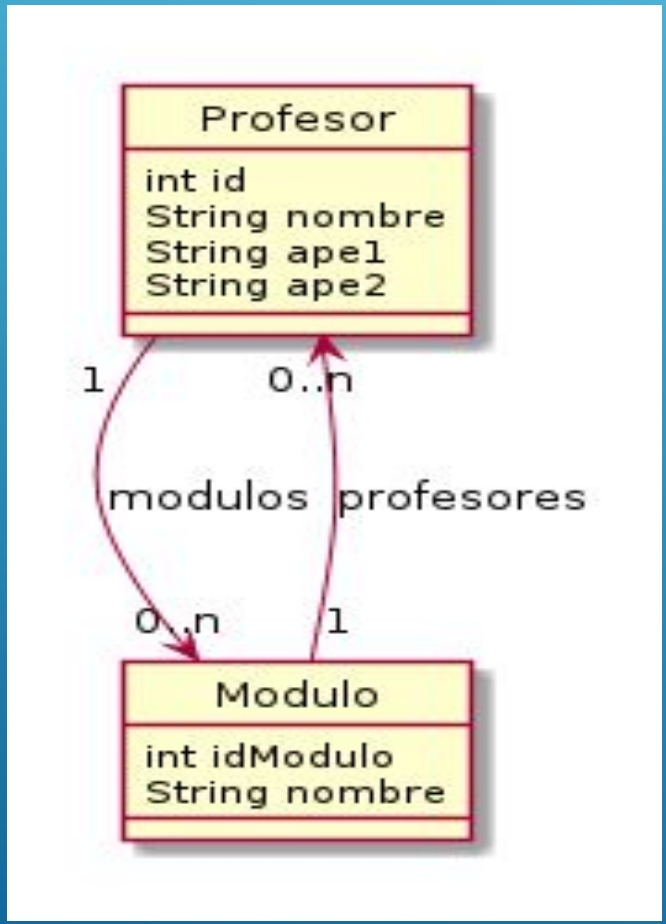
Relaciones Muchos a Muchos:

- ✔Dicha relación consiste en que un objeto A tenga una lista de otros objetos B y que a su vez este ultimo tenga un lista de objetos A.

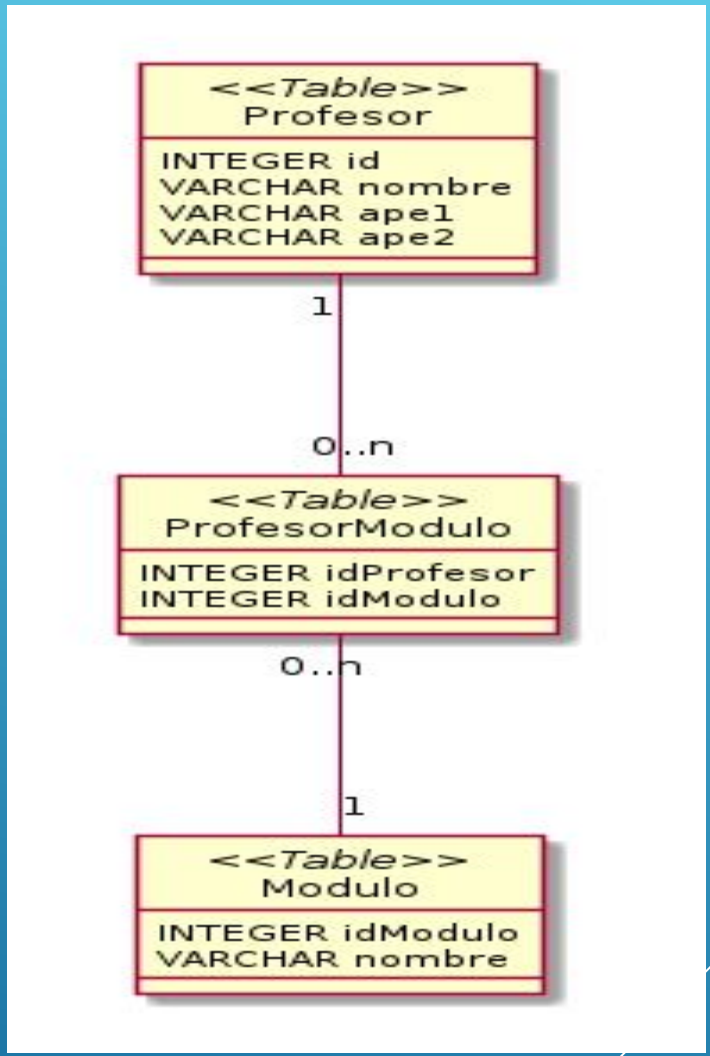
- ✔De forma que al persistir cualquier objeto también se persista la lista de objetos que posee.

HIBERNATE

Relaciones Muchos a Muchos:



UML



SQL

HIBERNATE

Relaciones Muchos a Muchos:

```java
@Entity
@Table(name="Profesor")
public class Profesor implements Serializable  {
    @Id
    @Column(name="Id")
    private int id;

    @Column(name="nombre")
    private String nombre;

    @Column(name="apellido_uno")
    private String ape1;

    @Column(name="apellido_dos")
    private String ape2;

    @ManyToMany(cascade = {CascadeType.ALL})
    @JoinTable(name="ProfesorModulo", joinColumns={@JoinColumn(name="IdProfesor")}, inverseJoinColumns={@JoinColumn(name="IdModulo")})
    private Set<Modulo> modulos;

    public Profesor(){
        modulos=new HashSet();
    }

}

```

@ManyToMany: Como su nombre indica le dice a ***Hibernate*** que la propiedad contendrá una lista de objetos que participa en una relación *muchos a muchos*.

HIBERNATE

Relaciones Muchos a Muchos:

```
@ManyToMany(cascade = {CascadeType.ALL})
@JoinTable(name="ProfesorModulo", joinColumns={@JoinColumn(name="IdProfesor")}, inverseJoinColumns={@JoinColumn(name="IdModulo")})
private Set<Modulo> modulos;
```

@JoinTable: Esta anotación contiene la información sobre la tabla que realiza la relación muchos a muchos

name: Nombre de la tabla que realiza la relación muchos a muchos. En nuestro ejemplo es ProfesorModulo.

joinColumns: Contiene cada una de las columnas que forman la clave primaria de esta clase que estamos definiendo. Cada columna se indica mediante una anotación ***@JoinColumn*** y en el atributo ***name*** contiene el nombre de la columna.

inverseJoinColumns: Contiene cada una de las columnas que forman la clave primaria de la clase clase con la que tenemos la relación. Cada columna se indica mediante una anotación ***@JoinColumn*** y en el atributo ***name*** contiene el nombre de la columna.

HIBERNATE

Relaciones Muchos a Muchos:

```java
@Entity
@Table(name="Modulo")
public class Modulo implements Serializable {
    @Id
    @Column(name="IdModulo")
    private int idModulo;

    @Column(name="nombre")
    private String nombre;

    @ManyToMany(cascade = {CascadeType.ALL},mappedBy="modulos")
    private Set<Profesor> profesores=new HashSet();

}
```

@ManyToMany: Indica que la propiedad contiene una lista de objetos que participan en una relación muchos a muchos.

mappedBy: Contiene el nombre de la propiedad Java de la otra clase desde la cual se relaciona con ésta. Al poner dicho atributo no es necesario incluir la anotación ***@JoinTable*** ya que dicha información se indica en el otro lado de la relación.

HIBERNATE

Relaciones Muchos a Muchos:

```
1  Profesor profesor1=new Profesor(11, "Isabel", "Fuertes", "");
2  Profesor profesor2=new Profesor(12, "Bruno", "Diaz", "");
3
4  Modulo modulo1=new Modulo(1, "Sistemas Operativos en Red");
5  Modulo modulo2=new Modulo(2, "Entornos de desarrollo");
6  Modulo modulo3=new Modulo(3, "Sistemas Informáticos");
7
8  profesor1.getModulos().add(modulo1);
9  profesor1.getModulos().add(modulo2);
10 profesor2.getModulos().add(modulo3);
11
12 modulo1.getProfesores().add(profesor1);
13 modulo2.getProfesores().add(profesor1);
14 modulo3.getProfesores().add(profesor2);
15
16 Session session=sessionFactory.openSession();
17 session.beginTransaction();
18
19 session.save(profesor1);
20 session.save(profesor2);
21
22 session.getTransaction().commit();
23 session.close();
24
```

De las líneas 8 a la 10 se añaden los módulos a los profesores.

De la línea 12 a la 14 se añaden los profesores a los módulos.

En las líneas 19 y 20 se persisten los profesores y automáticamente los módulos.

HIBERNATE

Relaciones (cascade):

✔*¿Qué significa dicho atributo?:* Su significado es indicar que debe de hacer ***Hibernate*** con las clases relacionadas cuando realizamos alguna acción con la clase principal.

✔Para esto contamos con 11 posibles valores de los cuales se debe de elegir uno de ellos al momento de su declaración.

HIBERNATE

Relaciones (cascade):

Valor	Descripción
none	No se realiza ninguna acción en los objetos relacionados al hacerlo sobre el principal
save-update	Si se inserta o actualiza el objeto principal también se realizará la inserción o actualización en los objetos relacionados.
delete	Si se borra el objeto principal también se realizará el borrado en los objetos relacionados.
evict	Si se llama al método Session.evict(Object objeto) para el objeto principal también se llamará para los objetos relacionados.
lock	Si se llama al método LockRequest.lock(Object objeto) 10) para el objeto principal también se llamará para los objetos relacionados.
merge	Si se llama al método Session.merge(Object objeto) con el objeto principal también se llamará para los objetos relacionados.
refresh	Si se llama al método Session.refresh(Object objeto) para el objeto principal también se llamará para los objetos relacionados.
replicate	Si se llama al método Session.replicate(Object objeto,ReplicationMode replicationMode) para el objeto principal también se llamará para los objetos relacionados.
all	Si se realiza cualquiera de las **anteriores** acciones sobre el objeto principal también se realizará sobre los objetos relacionados.
delete-orphan	Este atributo sólo se usa si el objeto relacionado es una colección. Indica que si en la colección del objeto principal eliminamos un elemento , al persistir el objeto principal deberemos borrar de la base de datos el elemento de la colección que habíamos eliminado.
all-delete-orphan	Es la unión de los atributos all y delete-orphan 11)

HIBERNATE

Relaciones (cascade):

- ✔*De los 11 posibles valores los que realmente importan, ya que incluyen la funcionalidad de trabajar con los objetos relaciones son:*
  - *none.*
  - *save-update*
  - delete
  - delete-orphan
- ✔Es normal que el atributo ***cascade*** tome alguno de los siguientes valores, según su importancia:
  - none.
  - All
  - All-delete-orphan
  - Save-update, evict, lock, merge, refresh, replicate.

HIBERNATE

Claves Primarias (PK):

```
1   @Entity
2   @Table(name="Profesor")
3   public class Profesor implements Serializable  {
4       @Id
5       @Column(name="Id")
6       @GeneratedValue(strategy=GenerationType.IDENTITY)
7       private int id;
8
9       @Column(name="nombre")
10      private String nombre;
11
```

- ✔ En la línea 6, se agregar una nueva anotación, la cual sirve para la generación de las ***PK.***
- ✔ ***@GeneratedValue:*** Dicha anotación indica a ***Hibernate*** deberá generar el valor de la ***PK.***
  - ***strategy:*** Este atributo indica el método en el que ***Hibernate*** debe generar la ***PK.***

strategy	Descripción
GenerationType.AUTO	Hibernate usará alguno de los siguientes métodos dependiendo de la base de datos. De esta forma, si cambiamos de base de datos, se seguirá usando la mejor forma de generar la clave primaria.
GenerationType.IDENTITY	Hibernate usará el valor de la columna de tipo autoincremento. Es decir que al insertar la fila la base de datos le asignará el valor. La columna de base de datos debe ser de tipo autonumérico
GenerationType.SEQUENCE	Se utiliza una secuencia como las que existen en Oracle o PostgreSQL . No es compatible con MySQL. La columna de base de datos debe ser de tipo numérico